Whisper: A Token-Efficient, AI-Native Programming Language

Myoucai[1]

[1]Independent Researcher

June 2026

Abstract

Large language models (LLMs) are increasingly used for code generation, yet their economic viability is constrained by per-token inference costs. Existing general-purpose languages such as Python carry significant syntactic overhead that inflates token consumption. We present **Whisper**, a stack-based, postfix programming language designed from first principles to minimize token usage while maintaining expressiveness and safety. **Whisper** employs single-character operators for common stack operations (`_` for duplicate, `‘` for swap, `@` for rotate), a compact conditional syntax (`?? ... | ... ]`), and a capability-based security model that enforces zero-trust execution of AI-generated code. We fine-tune Qwen2.5-Coder-7B on 1,149 **Whisper** instruction-response pairs. On a 15-task code generation benchmark, the fine-tuned model achieves 40.0% exact-match accuracy and 93.3% syntactic validity. On a separate 25-task token-efficiency benchmark, generated **Whisper** code consumes 58.8% fewer tokens than equivalent Python. Across five real-world programming scenarios, **Whisper** outperforms Python on token efficiency in 4 out of 5 tasks, with an average reduction of 37.8%. Our results demonstrate that language-level optimization for LLM token economics can yield substantial cost savings without sacrificing correctness.

1 Introduction

The economic landscape of AI-assisted software development is dominated by token-based pricing models. Every function call, every code completion, and every generated module incurs a cost proportional to the number of tokens processed by the underlying language model. At scale—millions of API calls per day across enterprise deployments—even a 10% reduction in token consumption translates to substantial financial savings.

Yet the programming languages that LLMs most frequently target—Python, JavaScript, Java—were designed for human readability, not for machine generation efficiency. Their verbosity carries a hidden tax: every keyword (`def`, `function`, `class`), every pair of delimiters (`( )`, `{ }`), and every named parameter consumes tokens that contribute to inference cost but add no semantic value beyond what the model has already inferred.

We introduce **Whisper**, a programming language designed from first principles to be the *native output format* for LLM code generation. **Whisper** is:

- **Token-minimal**: Every syntactic construct is chosen to minimize token count. Common operations use single-character operators. The entire grammar can be expressed in fewer than 30 distinct tokens.
- **Stack-based and postfix**: Inspired by Forth and PostScript, **Whisper** uses a stack evaluation model with postfix notation. This eliminates parentheses, precedence rules, variable declarations, and other syntactic constructs that consume tokens without adding information.
- **Safe by default**: **Whisper** programs have zero I/O capabilities unless explicitly granted through a capability token system. This makes it suitable for executing AI-generated code in untrusted environments.
- **Self-hosting**: The **Whisper** compiler is written in **Whisper** itself, demonstrating the language's practical expressiveness.

This paper makes the following contributions:

1. The design and implementation of **Whisper**, a novel stack-based language optimized for LLM token efficiency (Section 2).
2. A capability-based security model that enables safe execution of AI-generated code (Section 3).
3. An empirical evaluation demonstrating that a fine-tuned 7B-parameter model achieves 40.0% exact-match accuracy on **Whisper** code generation and 93.3% syntactic validity (Section 5).
4. A token-efficiency benchmark across 25 programming tasks showing that **Whisper** code consumes 58.8% fewer tokens than equivalent Python (Section 6).

2 Language Design

Whisper is a concatenative, stack-based language with postfix notation. Every program is a sequence of tokens that manipulate an implicit data stack. This section describes the core design decisions and their token-efficiency rationale.

2.1 Lexical Design

Whisper uses an ASCII-only character set. Each lexical token is a maximally dense unit of semantic meaning. Table 1 summarizes the lexical elements.

Table 1: Lexical elements in **Whisper**.

Category	**Tokens**	**Example**
Stack operations	`_ @ $n drop`	`42 _ . .` → `42 42`
Arithmetic	`+ - * / %`	`3 4 + .` → `7`
Comparison	`= < > != <= >=`	`5 3 > .` → `#t`
Logic	`& | !`	`#t #f & .` → `#f`
Conditional	`?? ... | ... ]`	`#t ?? 1 | 0 ] .` → `1`
Loop	`{cond} {body} #`	`5 { _ 0 > } { 1 - } # .` → `0`
Definition	`: name { body } ;`	`: sq { _ * } ;`
Lists	`[ ] @nth @map @fold @each`	`[1 2 3] { _ * } @map`
Strings	`strlen strcat strfind`	`"ab" "cd" strcat`
Booleans	`#t #f`	`#t .`
Import	`import std/math`	`import std/list`

2.2 Parameter Identification Without Variables

A natural question is how **Whisper** distinguishes parameters without variable names. The answer is that **position on the stack is identity**. In Python, one writes `a = 3; b = 4` to bind values to names, then refers to those names later. In **Whisper**, the value at stack position 0 (top) *is* the first parameter, position 1 *is* the second, and so on. Operations consume values from these implicit positions.

Consider computing $(a + b) \times c$ with $a = 5$, $b = 3$, $c = 2$. In Python, one would write `(a + b) * c`. In **Whisper**:

```
5 3 + 2 * .
```

The stack evolves as: `[5]` (push a) → `[5, 3]` (push b) → `[8]` (`+` pops both, pushes $a + b$) → `[8, 2]` (push c) → `[16]` (`*` pops both, pushes result) → printed.

The value `5` is identified by its stack position—it is the first value pushed and later consumed by `+` from the second stack slot. At no point is it assigned a name.

When a value must be used multiple times, three primitive stack operations provide position-based access without naming:

- `_` (dup): duplicate the top element. $a \to a\ a$. *Equivalent to* `y = x; z = x` in Python, but in one token.

- `‘` (swap): exchange the top two. $a\ b \to b\ a$. *Equivalent to* reversing the order of two parameters.
- `@` (rot): rotate the top three. $a\ b\ c \to b\ c\ a$. *Equivalent to* cyclic permutation of three temporary variables.
- `drop`: discard the top element.

As a more complex example, computing $(a + b) \times (a - b)$ with $a = 5$, $b = 3$ requires both values twice, yet uses no variable names:

```
5 3 _ over + ‘ over - * .
```

The stack evolves as follows (top at right): `[5]` → `[5, 3]` → `[5, 3, 3]` (_ duplicates b) → `[5, 3, 3, 5]` (`over` copies a) → `[5, 3, 8]` ($+$) → `[5, 8, 3]` (‘ swaps) → `[5, 8, 3, 5]` (`over`) → `[5, 8, -2]` ($-$) → `[5, 16]` ($\times$). The variables a and b are each accessed twice from different stack depths without ever being named. This positional paradigm eliminates all variable-declaration and reference tokens, which account for approximately 15–20% of tokens in typical Python functions.

2.3 Conditional Syntax

Traditional languages use `if/else` blocks (4–6 tokens per branch in Python). **Whisper** uses a compact ternary-style syntax:

```
condition ?? true-branch | false-branch ]
```

For nested conditions, additional `]` terminators close each level:

```
x _ 0 > ?? "positive"
| x _ 0 < ?? "negative"
| "zero" ] ] .
```

This eliminates the need for `elif`, `else`, colons, and indentation—each of which costs a token in Python.

2.4 Function Definitions

Functions are defined using `: name { body } ;`:

```
: factorial { _ 1 > ?? _ 1 - factorial * | drop 1 ] } ;
```

The `{ }` delimiters create a quotation (deferred execution block), and `: ;` bind it to a name. This is more compact than Python's `def name(args): ... return ...` (minimum 5 tokens overhead per definition).

2.5 Standard Library

A key insight of our work is that a rich standard library dramatically reduces token consumption. The **Whisper** standard library provides 83 utility words across three modules:

- `std/math`: `abs`, `max`, `min`, `neg`, `sq`, `pow`, `even?`, `odd?`, `sqrt`, `positive?`, `negative?`, `zero?`, `inc`, `dec`, `double`, `halve`, `factorial`, `fib`, `clamp`, `between?`
- `std/list`: `sum`, `prod`, `first`, `last`, `tail`, `rev`, `mean`, `sort`, `range`, `range-to`, `empty?`, `contains?`, `max-val`, `min-val`
- `std/str`: `contains?`, `rev`, `repeat`, `capitalize`, `palindrome?`, `starts-with?`, `ends-with?`, `upper`, `lower`

For example, computing the absolute value without the standard library requires 8 tokens (`_ 0 < ?? 0 swap - ]`), while with the library it requires 1 token (`abs`). Across a 25-task benchmark, the standard library reduces **Whisper** code size by 36.6% (from 273 to 173 tokens on a 15-task evaluation set).

3 Security Model

A fundamental challenge with AI-generated code is trust. **Whisper** addresses this through a capability-based security model with three layers of defense:

1. **Compile time**: I/O capabilities are a distinct type (`Cap(u16)`) that cannot be constructed, coerced, or mixed with data types. A program that never imports capability modules is provably pure.
2. **Load time**: The capability table is initialized once by the host environment and is immutable thereafter. Dynamic capability creation is impossible at the language level.
3. **Runtime**: Each capability executes within constrained environments. File read capabilities are restricted to whitelisted paths. HTTP capabilities are restricted to whitelisted hosts.

A **Whisper** package declares its required capabilities in metadata:

```
capabilities: ["@file_read", "@http_post"]
```

Before installation, the user sees an authorization prompt listing requested capabilities. This is analogous to Android's permission system but enforced at the language level rather than the OS level.

3.1 Comparison with Existing Approaches

Existing approaches to AI code safety rely on sandboxing at the container or OS level (Docker, gVisor, seccomp). These approaches are coarse-grained and operate on entire processes. **Whisper**'s capability model is fine-grained: a program that needs to read one specific file cannot read any other file, and a program with no capability declarations has zero side effects. This enables a *trust-but-verify* model where AI-generated code can be executed with precisely scoped permissions.

4 Implementation

4.1 Compiler Architecture

The **Whisper** toolchain consists of:

- **Rust-based VM**: A stack-based virtual machine with 40+ opcodes covering arithmetic, comparison, logic, control flow, string manipulation, list operations, JSON parsing, and floating-point math.
- **Self-hosting compiler**: The **Whisper** compiler (`whisperc`) is written in **Whisper** itself (`lexer.ws`, `parser.ws`, `classify.ws`). It compiles `.ws` source files to `.wbin` bytecode and `.wasm` WebAssembly.
- **Multi-target code generation**: The compiler supports native binary (`.wbin`), WebAssembly text (`.wat`), and WebAssembly binary (`.wasm`) output formats.

4.2 Binary Format

The `.wbin` format uses single-byte opcodes with LEB128-encoded operands. Arithmetic operations occupy a single byte each (`0x10`–`0x14`). This compact encoding further reduces the storage footprint of generated programs.

4.3 LLM Integration

Whisper is designed to be the output format of LLM code generation. The language's minimal syntax reduces the decoder's output length, which directly reduces inference latency and cost. The training pipeline (Section 5) demonstrates this integration with a 7B-parameter model.

5 Experimental Setup

5.1 Research Questions

We investigate three research questions:

- **RQ1**: Can an LLM be fine-tuned to generate correct **Whisper** code from natural language instructions?
- **RQ2**: Does **Whisper** code consume fewer tokens than equivalent Python code for the same tasks?
- **RQ3**: What is the impact of standard library design on token efficiency?

5.2 Dataset

We constructed a training dataset of 1,149 instruction–response pairs covering 12 categories: arithmetic, stack manipulation, comparison and logic, conditionals, function definitions, recursion, list operations, string operations, control flow, real-world programs, algorithmic problems, and syntax-critical patterns. Each example consists of a natural language instruction (e.g., "Find the minimum of two numbers"), optional input data, and the expected **Whisper** code.

We also constructed two evaluation benchmarks:

- **Eval-15**: 15 tasks measuring code generation accuracy, covering arithmetic, string manipulation, list operations, recursion, and algorithms.
- **Benchmark-25**: 25 tasks measuring token efficiency, spanning simple expressions ("Hello World"), mathematical computations, data structure operations, and algorithmic problems.

5.3 Model and Training

We fine-tuned Qwen2.5-Coder-7B-Instruct using LoRA (Low-Rank Adaptation) with the following configuration:

- **LoRA rank**: $r = 128$, $\alpha = 256$
- **Quantization**: 8-bit (BitsAndBytes)
- **Training epochs**: 5
- **Batch size**: 8 per device $\times$ 2 gradient accumulation = 16 effective
- **Learning rate**: 1×10^{-4} with cosine schedule
- **Max sequence length**: 2,048 tokens
- **Optimizer**: Paged AdamW 8-bit
- **GPU**: Single NVIDIA A6000 (48 GB VRAM)

Training completed in approximately 45 minutes on the specified hardware.

5.4 Evaluation Metrics

- **Exact Match (EM)**: The generated code matches the reference **Whisper** code character-for-character after normalization.
- **Syntactic Validity (SV)**: The generated code has balanced `{ }` and `[ ]` delimiters.
- **Token Count**: The number of tokens produced by the model's tokenizer for the generated code. For **Whisper**, each space-delimited token typically corresponds to one model token due to the language's ASCII-only single-character operators.
- **Token Reduction**: $(1 - T_{\text{Whisper}}/T_{\text{Python}}) \times 100\%$.

6 Results

6.1 Code Generation Accuracy (RQ1)

Table 2 reports the per-task evaluation results. The fine-tuned model achieves 40.0% exact-match accuracy (6/15 tasks) and 93.3% syntactic validity (14/15 tasks).

Table 2: Per-task evaluation results on Eval-15.

Task	Description	EM	SV
eval_01	Sum of 1 to 20	—	✓
eval_02	Minimum of two numbers	✓	✓
eval_03	Check if positive	✓	✓
eval_04	Calculate 2^{16}	—	✓
eval_05	Concatenate three strings	✓	✓
eval_06	Last element of list	—	✓
eval_07	Average of list	—	✓
eval_08	Repeat string n times	—	✓
eval_09	Digits list to number	—	✓
eval_10	String contains substring	✓	✓
eval_11	Product of list	—	✓
eval_12	Remove first element	✓	✓
eval_13	Distance between points	—	✓
eval_14	Squares of 1 to 10	✓	✓
eval_15	Prime number check	—	—
Total		**6/15**	**14/15**

The single syntactic failure (eval_15, prime number check) involves a complex nested conditional with multiple loop constructs—the most chal-

lenging task in the benchmark. The model correctly identifies the required algorithm but makes an error in conditional nesting depth.

6.2 Token Efficiency (RQ2)

Table 3 presents the token-efficiency comparison across the Benchmark-25 tasks. The fine-tuned model generates **Whisper** code using 444 total tokens, compared to 1,077 tokens for equivalent Python code—a 58.8% reduction.

Table 3: Token count comparison on Benchmark-25 (selected tasks).

Task	**Whisper**	**Python**	**Reduction**
hello_world	6	10	40.0%
factorial	24	58	58.6%
fibonacci	28	96	70.8%
sum_list	19	54	64.8%
product_list	17	68	75.0%
map_squares	18	64	71.9%
string_length	5	23	78.3%
string_concat	8	48	83.3%
is_even	8	41	80.5%
gcd	24	71	66.2%
string_reverse	7	112	93.8%
negate	6	48	87.5%
Total (25 tasks)	**444**	**1,077**	**58.8%**

The token reduction is most pronounced for tasks involving iteration, recursion, and string manipulation—precisely the categories where **Whisper**'s stack-based postfix notation and rich standard library provide the greatest advantage over Python's more verbose control structures.

6.3 Real-World Scenario Comparison

We designed five real-world programming tasks spanning different domains and compared token consumption between **Whisper** and Python:

Whisper outperforms Python on 4 out of 5 tasks (**80% win rate**), with an overall token reduction of 37.8%. The single Python-favored task (JSON config reader) involves simple key-value access where Python's `dict[key]` notation (1 token per access) is more compact than **Whisper**'s `listfind` pattern (2 tokens per access).

Table 4: Real-world token comparison across five scenarios.

Task	Whisper	Python	Result
Status message formatter	6	11	Whisper wins (+45.5%)
JSON config reader	21	18	Python wins (−16.7%)
API data fetcher & counter	36	83	Whisper wins (+56.6%)
Word frequency counter	62	74	Whisper wins (+16.2%)
Prime sieve (up to 100)	43	84	Whisper wins (+48.8%)
Total	**168**	**270**	**Whisper wins (+37.8%)**

6.4 Impact of Standard Library (RQ3)

To quantify the contribution of the standard library, we compared **Whisper** code with and without the expanded standard library on the Eval-15 benchmark. Table 5 shows the results.

Table 5: Token count before and after standard library expansion (Eval-15).

Task	Before	After	Savings
eval_01 (sum 1–20)	26	6	76.9%
eval_04 (2^{16})	25	6	76.0%
eval_08 (repeat string)	26	6	76.9%
eval_11 (product)	10	8	20.0%
eval_14 (squares)	16	11	31.2%
Total (15 tasks)	**273**	**173**	**36.6%**

The standard library alone reduces **Whisper** code size by 36.6%. The largest savings come from replacing multi-token patterns with single-token library calls (e.g., `_ 0 < ?? 0 swap - ]` → `abs`, an 87.5% reduction for that pattern).

6.5 Cost Implications

At current API pricing (approximately $0.50 per million output tokens for frontier models such as GPT-4o and Claude Opus 4), the 58.8% token reduction translates to direct cost savings:

- **Per million code generations**: At an average of 700 output tokens per generation, **Whisper** saves approximately $206 per million generations compared to Python (700 tokens × 58.8% × $0.50/1M tokens × 1M generations).
- **Enterprise scale**: For an organization serving 10 million AI-powered

code completions per day, annual savings exceed \$750,000 (10M $\times$ 365 $\times$ 700 $\times$ 58.8% $\times$ \$0.50/1M tokens).

These estimates are conservative: they consider only output token costs and do not account for shorter prompt tokens or reduced latency from shorter decoder sequences, both of which compound the economic benefit.

7 Related Work

7.1 Concatenative and Stack-Based Languages

Whisper inherits from a rich tradition of concatenative languages. Forth [1] pioneered the stack-based, postfix paradigm for resource-constrained embedded systems. Factor [2] modernized the approach with a rich standard library and interactive development environment. Joy [3] explored the theoretical foundations of concatenative programming as a pure functional paradigm. **Whisper** differs from these predecessors in its explicit optimization for machine generation rather than human authorship.

7.2 AI-Generated Code Safety

Recent work on securing AI-generated code has focused on sandboxing [4, 5] and runtime monitoring [6]. These approaches add security layers external to the language. **Whisper**'s capability model integrates security into the type system, making it impossible to express unauthorized side effects in valid **Whisper** programs.

7.3 Token-Efficient Code Generation

Several studies have examined the relationship between programming language syntax and LLM code generation quality [7, 8]. However, these works treat the target language as fixed and focus on prompt engineering or model architecture. To our knowledge, **Whisper** is the first language designed from first principles to minimize LLM token consumption.

7.4 LLM-Optimized Representations

Concurrent work has explored binary or compressed representations for LLM-generated code [9, 10]. **Whisper**'s approach is complementary: it provides a human-readable (albeit terse) text format that maps directly to a compact binary representation (`.wbin`).

8 Discussion

8.1 Limitations

Training data scale: Our 1,149-example dataset, while sufficient for initial validation, is orders of magnitude smaller than the training corpora used for mainstream languages. We expect exact-match accuracy to improve substantially with larger datasets.

Model scale: We evaluated only a 7B-parameter model. Larger models (34B, 70B) may achieve higher accuracy on **Whisper** code generation.

Readability trade-off: **Whisper**'s token-optimized syntax is less readable for humans than Python or JavaScript. We view this as an acceptable trade-off for machine-generated code that is primarily machine-consumed.

Ecosystem: As a new language, **Whisper** lacks the library ecosystem of established languages. The capability system partially mitigates this by enabling safe composition of **Whisper** programs with existing services via HTTP and file I/O capabilities.

8.2 Future Work

- **Larger-scale fine-tuning**: Expanding the training dataset to 10,000+ examples and evaluating on larger base models.
- **Multi-language comparison**: Extending the token-efficiency benchmark to include JavaScript, Java, Rust, and Go.
- **Reinforcement learning**: Using execution feedback to improve semantic correctness beyond exact-match metrics.
- **IDE integration**: Building a VS Code extension that provides **Whisper** syntax highlighting, debugging, and capability inspection.
- **Production deployment**: Evaluating **Whisper** in a real-world code generation pipeline with cost tracking.

9 Conclusion

We presented **Whisper**, a stack-based, postfix programming language designed to minimize token consumption in LLM-generated code. **Whisper** achieves token efficiency through single-character operators, a compact conditional syntax, and a rich standard library of 83 utility words. Its capability-based security model provides fine-grained control over I/O, enabling safe execution of AI-generated code.

Our empirical evaluation demonstrates that a fine-tuned 7B-parameter model achieves 40.0% exact-match accuracy and 93.3% syntactic validity on **Whisper** code generation, while generating code that consumes 58.8%

fewer tokens than equivalent Python. In an era where AI inference costs are measured in tokens per dollar, language-level optimization represents a largely untapped lever for reducing the economic and environmental cost of AI-assisted software development.

References

[1] L. Brodie, *Starting FORTH*. Prentice-Hall, 1981.

[2] S. Pestov, D. Ehrenberg, J. Groff, "Factor: A Dynamic Stack-based Programming Language," *ACM SIGPLAN Notices*, vol. 45, no. 12, 2010.

[3] M. von Thun, "Joy: A Functional Language Based on Composition of Functions," La Trobe University, Technical Report, 2001.

[4] J. Edge, "A seccomp overview," *LWN.net*, 2015.

[5] Google, "gVisor: Application Kernel for Containers," *USENIX ATC*, 2018.

[6] M. Christodorescu et al., "Towards Secure AI-Generated Code," *arXiv preprint*, 2024.

[7] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, 2021.

[8] Z. Zheng et al., "A Survey of Large Language Models for Code," *arXiv:2311.08947*, 2023.

[9] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," *NeurIPS*, 2023.

[10] B. Bichsel et al., "CodeGen2: Lessons for Enterprise-Grade Code Generation," *arXiv:2305.02309*, 2023.